Decision Analytics Journal 9 (2023) 100361

Contents lists available at ScienceDirect

Decision Analytics Journal

journal homepage: www.elsevier.com/locate/dajour

A Hybrid Particle Whale Optimization Algorithm with application to workflow scheduling in cloud–fog environment

Sumit Bansal *, Himanshu Aggarwal

Department of CSE, Punjabi University, Patiala, India

ARTICLE INFO

Keywords:
Whale Optimization Algorithm
Particle Swarm Optimization
Workflow scheduling
Cloud–fog computing
Simulation
Optimization

ABSTRACT

Workflow scheduling aims to optimize task allocation and execution in cloud–fog computing environments. Finding an optimal algorithm for workflow scheduling poses a significant challenge due to the complexity and variability of tasks and resources involved. Although meta-heuristic algorithms have been proposed to address these challenges, they often suffer from being trapped in local optima, failing to achieve the global optimal solution. Our work used a hybrid approach "Particle Whale Optimization Algorithm" (PWOA) to overcome the above-mentioned problem by combining Particle Swarm Optimization (PSO) and Whale Optimization Algorithm (WOA).

PSO's limitation in high-dimensional search spaces, characterized by slow convergence toward the global optimum, and the population-based WOA algorithm's limited exploration of search spaces are overcome by the proposed algorithm PWOA. The proposed algorithm PWOA overcomes the limitations of PSO and WOA.

The objective of the PWOA algorithm is to minimize the Total Execution Time (TET) and Total Execution Cost (TEC) associated with dependent tasks in a cloud–fog computing environment. To evaluate the performance of the PWOA algorithm, extensive simulations are conducted using test cases (30, 50, 100 and 1000) from five different scientific workflows: Cybershake, Epigenomics, Inspiral, Montage and Sipht. These workflows encompassed varying numbers of assigned tasks. The simulation results demonstrated that the proposed PWOA algorithm outperformed the standard PSO and WOA algorithms to improve TET and TEC across all the diverse workflows. This study also provides a comparative view of the effectiveness of the PWOA algorithm in enhancing workflow scheduling performance.

1. Introduction

The rapid growth of data-intensive applications and the increasing demand for real-time and low-latency services have led to the emergence of cloud and fog computing as complementary paradigms [1,2]. Cloud computing provides vast computing and storage resources in centralized data centres, while fog computing extends these capabilities to the edge of the network, enabling efficient data processing and localized service delivery [3]. The integration of cloud and fog computing architectures offers a distributed framework, enabling the efficient processing of data and applications closer to the edge [4,5]. Despite its potential benefits, workflow scheduling in cloud–fog environments remains a challenging task [6].

Scheduling involves the allocation and arrangement of multiple tasks [7] on shared resources [8,9], aiming to assign the appropriate resources to workflow tasks to achieve the completion of execution while satisfying various user-specified objective functions [10]. Workflow scheduling [11] constitutes a critical aspect of workflow management and falls into the category of NP-hard problems. Task mapping to all available virtual machines is a critical task and is employed to optimize the objectives of the proposed scheduling problem [12].

In existing studies, researchers have used meta-heuristic optimization algorithms to solve workflow problems in cloud–fog computing [13]. The meta-heuristic algorithms are easy to implement and have strong searching power [14]. They are more efficient in terms of time and cost. Genetic algorithm (GA) [15], a prominent meta-heuristic algorithm [16,17], has been extensively utilized in workflow scheduling research to minimize makespan. This choice is attributed to the algorithm's robustness and its capability to generate high-quality searches within a polynomial time frame, despite potentially requiring more time to find a solution. Author S. Pandey et al. [18] used the Particle Swarm Optimization (PSO) algorithm to schedule the applications to cloud resources and achieve minimum computation and data transmission costs. PSO is a fast optimized efficient algorithm, and it has an earlier convergence problem and gets trapped in local optimal solutions.

Overall, effective workflow scheduling in cloud–fog [19] computing environments is a complex and challenging task. Various scheduling

* Corresponding author.
E-mail addresses: pup.sumit@gmail.com (S. Bansal), himanshu.pup@gmail.com (H. Aggarwal).

https://doi.org/10.1016/j.dajour.2023.100361
Received 18 June 2023; Received in revised form 24 October 2023; Accepted 6 November 2023
Available online 8 November 2023
2772-6622/© 2023 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

issues, including local trapping, lack of balance in local and global optimal solution, low convergence speed, and high time consumption, have been identified in previous studies [20]. These findings indicate that a single scheduling algorithm is insufficient to address all these issues comprehensively.

To overcome these limitations and improve the performance of workflow scheduling, this study proposes a new meta-heuristic algorithm called Hybrid Particle Whale Optimization Algorithm (PWOA). The PWOA algorithm is developed through the hybridization of two existing standard algorithms, namely Particle Swarm Optimization (PSO) and Whale Optimization Algorithm (WOA). By combining these algorithms, the PWOA algorithm aims to exploit the strengths of each while minimizing their weaknesses.

In this study, the combination of PSO and WOA offers a synergistic approach. PSO excels at generating a set of feasible solutions, while WOA specializes in refining those solutions by searching for better alternatives. By leveraging the complementary strengths of both algorithms, the proposed PWOA algorithm enhances the quality of solutions by avoiding premature convergence and escaping local optima.

To evaluate the performance of the proposed PWOA algorithm, extensive simulations are conducted using a diverse set of five different workflows: Montage [21], Sipht [22], Cybershake [23], Inspiral [24], and Epigenomics [25]. These workflows are selected to represent various domains and encompass different number of tasks. The efficiency of the proposed algorithm is measured in terms of Total Execution Cost (TEC) and Total Execution Time (TET), which are then compared against standard algorithms. The simulation work is performed using the Workflowsim simulation toolkit [26], a widely adopted tool for workflow scheduling research. The simulation results demonstrate the superiority of the proposed PWOA algorithm over standard algorithms viz. PSO and WOA. The improvements achieved in terms of TEC and TET highlight the efficacy of the proposed algorithm in optimizing workflow scheduling performance.

This study addresses the limitations of existing scheduling algorithms by introducing the PWOA algorithm, which combines the strengths of PSO and WOA. The evaluation using diverse workflows showcases the algorithm's effectiveness in improving TEC and TET. The findings of this study contribute to the advancement of workflow scheduling research in cloud–fog computing environments, offering valuable insights for researchers and practitioners in the field.

1.1. Comparison of published papers in workflow scheduling: Analysing their strengths and limitations

In cloud–fog workflow scheduling, numerous researchers have extensively discussed research work. However, several unresolved issues persist concerning workflow scheduling algorithms. The major points presented in published papers within this field are compared and summarized in Table 1.

1.2. Limitations in existing papers

In the field of workflow management, task scheduling presents a notable obstacle. Service providers aim to optimize workflow scheduling to complete more tasks in less time, ultimately leading to higher revenue and improved profitability. Concurrently, users prioritize efficient resource utilization to reduce costs and enhance overall effectiveness, thereby ensuring access to services at affordable prices. However, existing research papers exhibit certain limitations. In the past, research studies have examined algorithm performance by focusing on only a limited number of scientific workflows. In our work, we focus on five scientific workflows, including Cybershake and others. While existing studies emphasize a limited number of performance metrics, our focus expands to a broader range of performance metrics. The current studies do not include testing with various test cases, such as 30, 50, 100, and 1000 iterations. By addressing these shortcomings, it becomes possible to enhance the efficiency and effectiveness of workflow task scheduling. Furthermore, the combination of PSO and WOA has not been explored in the previously discussed literature. These limitations and observations have motivated us to evaluate a new hybrid algorithm, named as PWOA, which harnesses the synergistic capabilities of both approaches.

1.3. Research highlights

The major highlights of this research work, which have been derived to overcome scheduling issues, are:

- A hybrid meta-heuristic algorithm for solving the workflow scheduling problem.
- Addressing the issue of local optimal solutions through algorithm hybridization.
- Comparative analysis of results across five workflows with varying numbers of test cases.

1.4. Novelty of research

The existing research papers, the majority of researchers who evaluated outcomes for only a limited number of scientific workflows and single units of iterations, our proposed system comprehensively compares results across all five scientific workflows cybershake etc. and at varying iterations to assess performance and reliability.

Furthermore, prior studies have not thoroughly examined various performance measures, such as the effective allocation of tasks, time, and cost. Our proposed system seeks to address this gap by considering these performance measures in evaluating outcomes.

1.5. Structure of the paper

The remainder of this paper is structured as follows. Section 2 explains the "Background" of the paper such as fitness function, standard PSO algorithm, gbest and pbest values, position and particle velocity, WOA algorithm, encircling prey, exploitation phase, and exploration phase of the paper. Section 3 named "Proposed PWOA algorithm" explains the approach to solve the formulated problem. Experimentation and comparison of results are provided in the "Performance Evaluation" section of the paper. Finally, the conclusion and the future works are presented in the "Conclusion and future works" section.

2. Background

The fitness function is a key component in optimization algorithms used for solving various problems, including workflow scheduling. Next, section discusses the fitness function of the algorithm.

2.1. Fitness function (FF)

The algorithm's parameters are optimized with the use of Fitness Function. This study evaluated two fitness functions. TEC and TET are calculated by F1 and F2 Respectively. Table 2 explains the notations.

Eqs. (1) and (5) define TEC and TET, respectively. The authors of this research article, minimized the TEC and TET specified by the Fitness Functions.

$$\text{Fitness (F1)} = \text{Total execution cost (TEC)} \tag{1}$$

$$\text{TEC} = \sum_{j=0}^{nVM-1} C\ j * \text{LET}\ j - \text{LST}\ j \tag{2}$$

The computation of TEC is shown in Eq. (2). The j_{th} virtual machines TEC is the product of Least Start Time (LST), Least End Time (LET) and Cost (C[j]) of the virtual machines. C[j] is the j_{th} virtual machine's execution cost per unit time period. For all Virtual Machines (VMs), the value of C is assumed to be 1.

$$\text{LET}\ j = \max(\text{ET[j]}) \tag{3}$$

Table 1
A comparative analysis of existing published works in the field of cloud–fog computing workflow.

References	Name of algorithm	Workflow used	Performance metrics	Strengths	Limitations and open challenges
Tong et al. [27]	BBO and HMBBO	4	Makespan	Fast convergence rate compared to BBO, PSO and HEFT	The workload over resources and cost is not considered
Ijaz et al. [28]	Multi-Objective Optimization algorithm	2	Energy and Makespan	Reduced energy consumption up to 50% when compared to EM-MCC, MOPT, and PEFT	The results compared with Cybershake and Montage workflows only
Hammed and Arunkumar [29]	Cost-effective algorithm	3	Cost	Reduced execution cost Comparison of QoS parameters with PSO and GA	The workload over resources is not considered, and no workflow used
Bothra et al. [30]	Cost-effective hybrid genetic algorithm	4	Cost	CHGA exhibits cost savings of 14.58%, 11.40%, 11.75% and 9.78% in comparison to standard ACO, PSO, CEGA and CLGA	Focused on cost reduction only other parameters are not considered
Alsmady et al. [31]	Memetic Algorithm (MA) [32]	0	Cost and Deadline	Reduced makespan and outperforms than GA and PSO algorithms	Solely focused on makespan, without considering other parameters
Wang et al. [33]	PSO and LOA	3	Makespan	Outperforms GA with respect to budget constraints in large-scale applications	The consideration of the workload over resources is omitted
Arora and Banyal [34]	PSO–GWO algorithm	4	Time and Cost	Decreased TET and TEC when compared to PSO and GWO algorithms	Workload is not taken into account
Chen et al. [35]	Population-based approach	0	Makespan and Load balancing	Reduced total time and cost by allocating tasks to their suitable resources	No workflow is used for comparison
Sarma [36]	Dragon Levy updated squirrel algorithm (DLU-SA)	0	Cost	Scheduling algorithm for workflow is employed to enhance resource utilization, focusing on factors such as makespan, deadlines, and real-time applications	Results are compared with four scientific workflows for cost only
Bisht and vampugani [37]	Min–min algorithm	0	Cost, makespan, energy, and load balancing	Minimized makespan, energy consumption, improvement in load balancing and costs when compared to the Min–Min and ELBMM algorithm	No workflow used and workload over resources is not considered
Chen et al. [38]	Energy aware scheduling algorithm	1	Energy	Offers an improvement in terms of schedule length, optimal solution ratio, and execution time	The load over resources is not considered and the results are compared with one scientific workflow only
Mokni et al. [39]	Multi-objective scheduling approach	0	Makespan and cost	Reduced makespan by 48% compared to the EM-MOO approach and 41% compared to the MAS-GA	Focused on cost and makespan reduction only
Chen et al. [40]	Novel energy coefficient-based algorithm	2	Energy consumption	Improvement in energy consumption and data dependency management	Focused solely on energy consumption and compared the results with two workflows only
Farhat et al. [41]	Reinforced R-learning based Decision model	0	Minimize cloud load	Reduced cloud processing load from 100 to 30%	No workflow is utilized, only cloud server load is considered as objective
Liu et al. [42]	Energy-conscious genetic algorithm	2	Energy and Makespan	Used fitness function and GA as scheduling approach to minimize energy and makespan	Focused on Energy and makespan. Only two workflows are used for comparison
Arora and Banyal [43]	PSO–GWO algorithm	4	Time and Cost	PSO–GWO algorithm reduced the average TEC and average TET compared to the standard PSO and GWO algorithms	Workload over resources is not considered
Stavrinides and Karatza [44]	Hybrid-EDF	0	To lower the deadline's miss ratio	Heuristic scheduling algorithm reduced the deadline miss ratio on average by 76.69% as compared to the baseline policy	The sole criteria for evaluation is monetary cost

Table 2
Notations used in this section along with their corresponding explanations.

Notation	Explanation
VM	Virtual machine
nVM	Total number of virtual machines
LST	Least start time
LST[j]	Least start time of j_{th} virtual machine
LET	Least end time
LET[j]	Least end time of j_{th} virtual machine
TET	Total completion time of tasks
TET[j]	Total completion time of j_{th} virtual machine
CT	Completion time
CT_i	Completion time of job T_i's
VM_j	For j_{th} number of virtual machines
T_i	i_{th} number of tasks
w	Represent workflow
WPT_i	Waiting time of predecessor tasks for i_{th} task
MIPS	Million instructions per second
SZ_{T_i}	Length of task T_i in million instructions
$Num(PE_j)$	Number of cores assigned to j_{th} virtual machine
PE_{tUnit}	size of each core in MIPS
ET	Execution time
ET[j]	Execution time of j_{th} virtual machine

$$LST\ j\ = \min(ET[j]) \tag{4}$$

The LET and LST are calculated by using Eqs. (3) and (4), respectively. The maximum ET of the j_{th} VM is represented by LET of that VM. LST is the minimal ET of the j_{th} virtual machine.

The ET of the jobs performed on VM_0 and VM_1 is shown on the x-axis. VM_0 and VM_1 are shown on the y-axis. Tasks T_3, T_5 and T_6 are running on virtual machine VM_1. Virtual Machine ($VM_{0)}$ is utilizing to complete tasks T_1, T_2, T_4 and T_7. LST (LST [1]) represent the beginning of T_3 on VM_1, whereas LET represents the end of T_6 on VM_1 (LET [1]).

The execution cost of VM_1 is calculated by multiplying the total difference between LET [1] and LST [1] with the execution cost(C [1]).TET is used to create another fitness function (F2), as shown in the equation below. As shown in Eq. (6) below, the TET is the maximum completion time of workflow task T_i.

$$\text{Fitness (F2)} = \text{Total Execution Time (TET)} \tag{5}$$

$$TET_w = \max\ \ CT_i \eth i = 1; 2; 3; 4 \ldots ::m \tag{6}$$

In the workflow, CT_i is representing the completion time of job T_i. TET is the completion time of the tasks and 'w' refers to workflow. If tasks exhibit dependencies, the waiting time of predecessor tasks is also taken into account. The Eq. (7) represents the completion time CT_i.

$$CT_i = \begin{matrix} T \\ ET_i \\ WPT_i + ET_i \end{matrix} \quad \begin{matrix} \\ \text{iff pred}(T_i) = \varnothing \\ \text{iff pred}\ \ T_i\ \ `\ \varnothing \end{matrix} \tag{7}$$

As shown in Eq. (8) below, the waiting time of assigned task T_i is the maximum completion time of the entire process predecessor tasks.

$$WPT_i = \begin{matrix} T \\ 0 \\ \max(CT_i) \end{matrix} \quad \begin{matrix} \\ \text{iff pred}(T_i) = \varnothing \\ \text{iff pred}\ \ T_i\ \ `\ \varnothing \end{matrix}$$

$$ET_{i;j} = \frac{SZ_{T_i}}{Num\ \ PE_j\ \ * PE_{Unit}} \tag{8}$$

The above equation is used to measure the ET of task T_i on virtual machine VM_j, in which SZ_{T_i} is the length of task T_i in million instructions, $Num(PE_j)$ is the total number of cores assigned to the virtual machine VM_j, and PE_{Unit} is the size for each and every core in MIPS. The computation of TET is shown in Fig. 1. When task T_7 completes its execution, the workflow's TET occurs.

Table 3
Notations used in this section along with their corresponding explanations.

Notation	Explanation
$\dot{A}$	Coefficient vector for convergence
$\dot{C}$	Used to avoid the trapping of search agents in the local optimum solution
$\dot{D}$	Used to model the process around the prey in surroundings mathematically
$\dot{X}$	The position vector of the current search agent
p	Random number
b	Constant value
l	Randomly generated number
N	Total number of iterations
iter	Current iteration
pbest	Particle best
gbest	Global best

The Sections 2.2 and 2.3 discusses the standard PSO and WOA algorithms. It explains step-by-step interpretation and implementation of global and particle best values, particle velocity, next position, encircling prey, bubble net attacking method etc. of algorithms respectively.

2.2. Global best and particle best values

PSO algorithm [45] calculates global best (gbest) and particle best (pbest) values. In the PSO algorithm, "pbest" represents the personal best solution found by an individual particle, reflecting its best performance within the search space. On the other hand, "gbest" stands for the global best solution attained by considering the best-performing particle among all individuals in the swarm. Table 3 illustrates the list of notations used in this section along with explanations.

In each iteration, the algorithm keeps track of each particle and traces the 'gbest' and 'pbest' values. Every solution is compared based on its fitness value calculated by the FF illustrated in Section 2.1. The solution having smallest fitness value ever achieved in all iterations performed is referred to as 'gbest' value. The best solution achieved by each particle ever is referred to as 'pbest' value. Next, Algorithm 1 shows the pseudo-code to calculate the 'pbest' and 'gbest' positions of particles.

Algorithm 1. Calculate pbest and gbest position of particle

```
Input: Particle
Output: pbest and gbest values
Set: pbest[0 : totalparticle -1] ← ∞; gbest ← ∞; i←0;
while i<totalparticle do
    if i=0 then
        pbest[i] ← particle[i]; gbest← particle[i];
    end if
    while j <= particle[i].totalPosition
     if pbest[i].TEC>particle[i].TEC then
        pbest[i] ← particle[i]
     end if
     if gbest.TEC>pbest[i].TEC then
        gbest ← particle[i]
     end if
     i = i+1
    end while
end while
```

2.2.1. Update particles velocity and next position

With the course of iteration of the PSO algorithm, position and velocity are two factors that are updated. Particles move towards a more optimal position with each iteration based on velocity. The

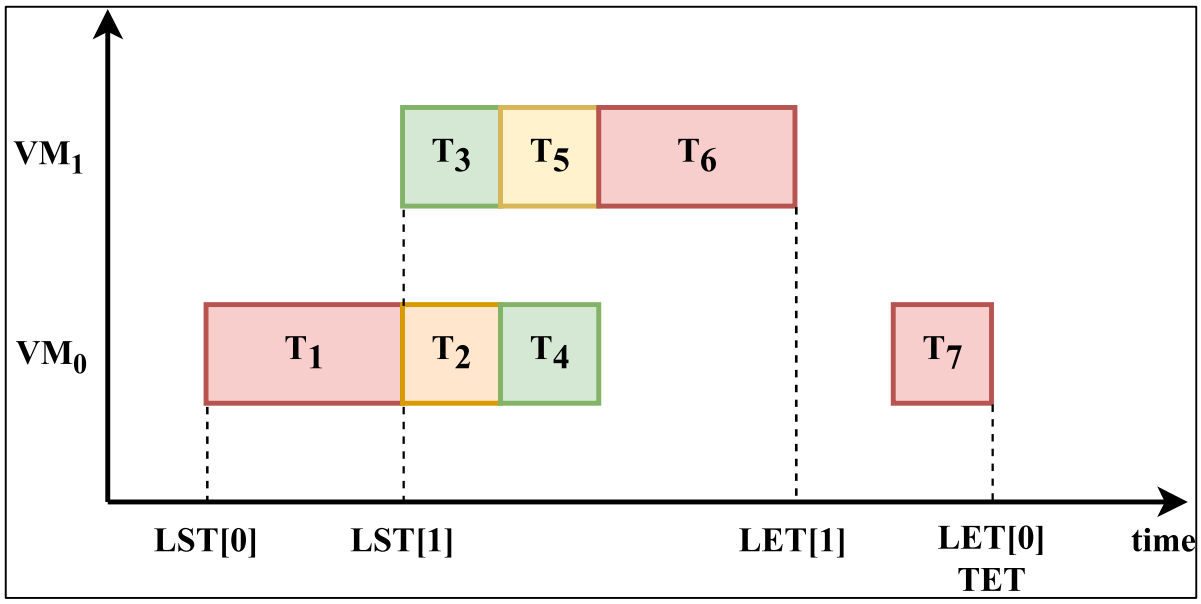

Fig. 1. An example to calculate TET.

Table 4
Parameters used during implementation for PSO algorithm.

Parameter	Value	Explanation
N_{pa}	25	Size of the population of the particles that are exploring for the solution by being updated by equations
N_i	500	Maximum number of iterations for the solution exploration
I_w	0.1	Factor to control cognition of particle
r_1, r_2	[0,1]	Factor to control social influence of swarm
c_1	2	Inertia makes the particle move in the same direction and with the same velocity
c_2	2	Random variable to be assigned in [0,1] interval on every iteration
R_e	15	Number of experiments to be repeated for calculation of average solution

velocity of each particle is determined by the influence of the 'gbest' and its 'pbest' value.

$$V_i(\text{iter}+1) = w * V_i(\text{iter}) + c_1 * r_1 * \text{pbest} - x_i(\text{iter}) + c_2 * r_2 * \text{gbest} - x_i(\text{iter}) \quad \ldots\ldots\ldots \quad (9)$$

The i_{th} particle's velocity at iteration 'iter + 1' is represented in Eq. (9). The inertia weight, or I_w, is used to prevent an infinite increase in particle velocity. The particle's present position is supplied via x_i(iter). The initial values for the coefficients c_1 and c_2 are shown in Table 4 below. The random values between [0–1] are r_1 and r_2.

Algorithm 2 and 3 shows the steps to update particle velocity and position.

The simulation parameters that were taken during the evaluation of the suggested work are displayed Table 5 (below). During the simulation, this uses 2 cloud servers and 2 fog nodes. In comparison to fog cloud servers have less processing power and are cheaper. The amount of MIPS (shown below) has been applied across various servers to execute the tasks.

2.3. Encircling prey

The WOA algorithm, proposed by Mirjalili and Lewis [46], employs each whale as a search agent, representing a potential solution. In certain scenarios, it may demonstrate superior performance compared to other meta-heuristic algorithms. The mathematical equations used in this algorithm are elaborated upon below. Table 6 presents the initial parameter values used in the simulation work of the WOA algorithm.

During the process, humpback whales encircle the prey. The best solution obtained so far in all iterations is treated as leader whale. Other search agents, whales are updating their position based on leader whale. This behaviour is represented in Eq. (10) and (11)

$$\dot{D} = \left|\dot{C} \cdot \dot{X}^{*}(\text{iter}) - \dot{X}(\text{iter})\right| \quad (10)$$

$$\dot{X}(\text{iter}+1) = \dot{X}^{*}(\text{iter}) - \dot{A} \cdot \dot{D} \quad (11)$$

The current iteration is represented by iter, $X^{'}*$ refers to the best solution ever achieved (leader whale), $\dot{X}$ is the position vector of the current search agent. $\dot{D}$ is used to model the process around the prey

Table 5
Simulation parameters during the evaluation.

Parameter	Value	
Number of tasks	20–1000	
Number of VM	4	
Number of cloud servers	2	
Number of fog nodes	2	
MIPS (Cloud/ Fog)	1600	1300
Number of processors	1	
RAM	512	
Virtual Machine policy	Time shared/Space shared	

Table 6
WOA algorithm parameters.

Parameter	Value	Explanation
S_a	25	Size of the population of the whales that are exploring for a solution by being updated by equations
N_i	500	The maximum number of iterations for the solution exploration
R_e	15	Number of experiments to be repeated for calculation of average solution
r_1, r_2	[0–1]	Random variable to be assigned in [0, 1] interval on every iteration
b	1	b is a constant for defining the shape of the logarithmic spiral

Algorithm 2. Update position of particle

Input: Updated particle velocity
Output: Updated particle position
Set: pbest ← null; gbest ← null;
while i<particleLength do **(i=index of particle)**
  while j<totalPosition do
    particle[i].pos[j] = particle[i].pos[j] + particle[i].vel[j];
    j+=1
  end while
  i+=1
end while

Algorithm 3. Update velocity of particle

Input: Particle velocity value
Output: Updated particle velocity value
Set: pbest ← null; gbest ← null; r_1,r_2 ← rand[0,1]; c_1,c_2 ← 2; I_w ← 0.1
while i<totalparticle do **(i=index of particle)**
  while k<totalIteration do **(k=index of iteration)**
    particle[i].vel[k+1] ← w * (particle[i].vel[k] + c_1 * r_1 * ((particle[i].pbestpos[k] – particle[i].pos[k])) + c_2 * r_2 * ((gbest.pos[k] – particle[i].pos[k])))
    k=k+1
  end while
  i = i+1
end while

in surroundings mathematically. This value is determined by the best position and current search agent position. $\dot{A}$ and $\dot{C}$ is the coefficient vectors that can be calculated as follows in Eqs. (12) and (13)

$$\dot{A} = 2\dot{a}\cdot r_1 - \dot{a} \tag{12}$$

$$\dot{C} = 2\cdot r_2 \tag{13}$$

The coefficient vector $\dot{A}$ is responsible for convergence $\dot{C}$ is used to avoid the trapping of search agents in the local optimum solution. The above equation is used to evaluate the linear value reduction from 2 to 0. The range of random variables r_1 and r_2 is between 0 to 1.

2.3.1. Bubble net attacking method (exploitation phase)

Two approaches are designed mathematically to observe and analyse the humpback whale bubble-net behaviour and shown in Eq. (14).

$$\dot{X}(\text{iter}+1) = \begin{cases} \dot{X}^*(\text{iter}) - \dot{A}\cdot\dot{D} & \text{if } p < 0.5 \\ D' * e^{bl} * \cos(2\ \ l) + \dot{X}^*(\text{iter}) & \text{if } p \geq 0.5 \end{cases} \tag{14}$$

Where 'p' is a random number in the range [0,1]. If $p < 0.5$ then the position will be updated based on the distance between best and current search agent. For $p \geq 0.5$, a spiral equation is introduced mentioned in the second part of the above equation. A spiral equation is proposed to mimic the humpback whales helix-shaped movement. The constant 'b' is used to define the shape of a logarithmic spiral. 'l' is a randomly generated number within the range of [−1, 1], used for step-by-step multiplication. Where, "D" and 'l' will be calculated using Eqs. (15) and (16).

$$D' = \delta\dot{X}^*(\text{iter}) - \dot{X}(\text{iter})\delta \tag{15}$$

$$l = \ \ a_2 - 1\ \ * r + 1 \tag{16}$$

'r' is the random variable between [0,1] whereas variable a_2 decreases linearly from −1 to −2 with iterations according to Eq. (18).

2.3.2. Search for prey (exploration phase)

To provide exploration and exploitation, Eq. (17) decreases the controlling parameter from 2 to 0 over the course of each iteration, and Eq. (18) decreases a_2 from −1 to −2.

$$a_1 = 2 * (1 - \frac{\text{iter}}{N}) \tag{17}$$

$$a_2 = -1 + \text{iter} * \frac{-1}{N} \tag{18}$$

'N' is the total number of iterations and 'iter' is the current iteration. The steps to determine the leader position i.e. best solution in whales is shown in algorithm 4. Algorithms 5, 6 shows the process of attacking and haunting the prey respectively.

Algorithm 4. Calculate leader position

Input: Whales
Output: leader position
Set: leaderpos ← null
while i<totalparticle do
  if i=0 then
    leaderpos ← gbest
  end if
  if whales[i].TEC < leaderpos.TEC then
    leaderpos ← whales[i]
  end if
  i = i+1
end while

Algorithm 5. Calculate A, C, a_1, a_2 and l values

Input: A, C, a_1, a_2 and l
Output: Updated A, C, a_1, a_2 and l
Set: r_1 ← rand [0,1], r_2 ← rand [0,1]
while i< N do
  A ← 2 * a_1 * r_1 - a_1
  C ← 2 * r_2
  a_1 ← 2 - i * (2 / N)
  a_2 ← -1 + i *((-1) / N)
  l ← [a_2 -1] * rand [0,1] + 1
  i = i+1
end while

3. Proposed PWOA algorithm

The article proposed a meta-heuristic algorithm known as PWOA, which is a combination of PSO and WOA algorithms. The hybridization of Particle Swarm Optimization (PSO) and Whale Optimization Algorithm (WOA) can yield several benefits. By combining the strengths

of both algorithms, the hybrid approach aims to enhance optimization performance and overcome individual algorithm limitations. PSO is known for its efficient exploration of search spaces and rapid convergence.

The initial phase of the algorithm involves generating a population of particles in a random manner, with each individual particle serving as a potential solution to the workflow issues. As a result, the PSO algorithm were executed half of the total number of iterations selected by the user, providing us with the population's best random solution. The outcome of these half iterations using the 'gbest' solution of the PSO algorithm is then employed as the leader whale in the WOA algorithm. WOA excels in global exploration and exploitation, utilizing the remaining half of the iterations to generate the final outcome of the proposed algorithm.

The hybridization process can lead to improved convergence rates, as PSO's quick convergence helps guide WOA towards promising regions of the search space. Additionally, the hybrid approach can provide better balance between local and global search, allowing the algorithm to escape local optima more effectively. Overall, the synergy between PSO and WOA in a hybrid setting can lead to increased optimization efficiency, better solution quality, and improved performance across a wider range of optimization tasks and scenarios. In this work, five scenarios are formed using the Montage, Cybershake, Epigenomics, Sipht, and Inspiral scientific workflows. These workflows are tested with various task numbers; for instance, Montage is tested with 20, 60, 100 and 300 tasks.

4. Simulation environment

The experimental outcomes of all different scenarios, namely Cybershake etc., are discussed here. These scenarios are available with varying ranges of tasks. For example, Cybershake has 30, 50, 100, and 1000 tasks. The experimental work was conducted using a MacBook with a 1.8 GHz Dual-Core Intel Core i5 processor, 8 GB 1600 MHz DDR3 RAM, and the MacOS Monterey Operating System version 12.3.1 (21E258). Implementation work was carried out using Workflowsim, an extension of the cloudsim simulation toolkit. The next subsection shows the TEC and TET result and outcomes as percentages with respect to algorithms and number of iterations.

Table 7
PWOA proposed algorithm parameters.

Parameter	Value	Explanation
N_p	25	Size of the population of the search agent that is exploring for a solution by being updated by equations
P_l	N_t	Maximum number of Iterations for the solution exploration
N_i	500	Maximum number of Iterations for the solution exploration
I_w	0.1	Inertia makes the particle/whales move in the same direction and with the same velocity
R_e	15	Number of experiments to be repeated for calculation of average solution
C_1	2	Factor to control cognition of search agent
C_2	2	Factor to control social Influence of population

5. Performance assessment

To evaluate the proposed algorithm, the results are compared with WOA and PSO standard algorithms. The scientific workflow, such as Cybershake, consist of four scenarios each. which is available with 30, 50, 100 and 1000 iterations. The algorithms performance is measured by minimizing the TET and TEC through workflow scheduling. In the first phase, the algorithm's parameters are initialized as shown in Table 7. The proposed algorithm starts with a randomly generated population of particles, where each particle represents a solution to the workflow issues. The PSO algorithm is executed for half of the total number of iterations, denoted as $\frac{n}{2}$, to implement the population's random solution. The gbest outcome of $\frac{n}{2}$ iterations of PSO algorithm are utilized as a leader whale by WOA algorithm. The remaining half of the iterations $\frac{n}{2} + 1$ to n are executed with that initialized leader whale via WOA algorithm. The results generated by the WOA algorithm are used as the final outcomes of the proposed algorithm. To improve the accuracy of the results, we performed multiple iterations i.e., 15. Because, in optimization algorithms, results tend to approach the optimal solution gradually, hence the need for iterations. In this work, we utilized input parameters to control the number of iterations. These parameters guided the algorithms to iterate and evaluate the results. Upon completing the iterations, the algorithm selected the best result obtained during the process as its final output.

After evaluating the algorithm results, it is evident that the performance of the new algorithm is better to that of the standard algorithms. This becomes possible due to the tendency of the PSO algorithm to become trapped in locally optimal solutions when dealing with complex and challenging NP-hard problems such as workflow scheduling. Consequently, the proposed PWOA algorithm outperforms the PSO approach.

Therefore, after saving the results obtained from the PSO algorithm, we input them into the WOA algorithm. This further validates the outstanding TEC and TET results attained by the proposed PWOA algorithm. The proposed algorithm and its corresponding flowchart are presented in Algorithm 7 and Fig. 2, respectively.